INS Lab Exp-1

Aim:

Perform encryption, decryption using the following Substitution techniques
(i) Ceaser cipher
(ii) Playfair cipher
(iii) Hill Cipher

(i) Ceaser cipher

Description:

Caesar cipher:
Encryption:
Simple substitutional cipher that shifts each letter of the plan text by fixed number of positions down the alphabet
Ex: shift 3
A -> D
B -> E
Decryption:
Involves shifting the cipher text letters back by the same number of positions
Ex: shit 3
KHOOR -> HELLO
Algorithm:
Each plain text letter p substitute cipher text letter c
C = E(P,K) MOD 26 = (P+K) % 26 -> Encryption
P = D(C,K) MOD 26 = (C-K) % 26 -> Decryption

Implementation:

```
def encrypt(text, s):
    result = ""
    for i in range(len(text)):
        char = text[i]
        if char.isupper():
            result += chr((ord(char) + s - 65) % 26 + 65)
        elif char.islower():
            result += chr((ord(char) + s - 97) % 26 + 97)
        else:
            result += chr((ord(char) + s - 32) % 95 + 32)
    return result
text = "cbit ACADEMY @ "
```

```
s = 4
print("Text  : " + text)
print("Shift : " + str(s))
print("Cipher: " + encrypt(text, s))
```

Output:

```
cselab8@cselab8-OptiPlex-3060:~/014$ python3 exp1/ceaser.py
Text  : cbit ACADEMY @
Shift : 4
Cipher: gfmx$EGEHIQC$D$
```

Results:

The Caesar cipher algorithm was implemented successfully and tested with a sample plaintext. The encryption process correctly shifted each character by the specified key value, demonstrating the working of substitution-based encryption. The program handled uppercase letters, lowercase letters, and special characters correctly.

Conclusion:

The Caesar cipher is a simple and effective method for understanding the basics of encryption and decryption. It demonstrates how plaintext can be converted into ciphertext using a fixed shift key. Although it is easy to implement, it provides very low security and is mainly used for educational purposes rather than secure communication.

(ii) Playfair cipher

Description:

Playfair cipher

Playfair Cipher is a classical symmetric-key encryption technique used to secure text.
It encrypts pairs of letters (digraphs) instead of individual letters, making it more secure than a simple substitution cipher.

- A 5 × 5 key matrix is generated using a keyword by removing duplicate letters.
- The letters I and J are considered as a single character to fit the 25-letter matrix.
- Before encryption, the plaintext is converted into digraphs by removing spaces and special characters, replacing J with I, inserting X between repeated letters, and adding X if the message length is odd.
- Encryption is performed using three rules:
    - Same Row: Replace each letter with the letter to its right.
    - Same Column: Replace each letter with the letter below it.
    - Rectangle Rule: Replace each letter with the letter in the same row but in the other letter's column.
- Decryption applies the reverse of these rules to recover the original plaintext.
- The Playfair Cipher provides better security than a simple monoalphabetic substitution cipher by reducing the effectiveness of frequency analysis.

Implementation:

```
def generate_key_table(key):
    key = key.upper().replace("J", "I")
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
    seen = set()
    table = []
    for ch in key + alphabet:
        if ch.isalpha() and ch not in seen:
            seen.add(ch)
            table.append(ch)
    return [table[i:i + 5] for i in range(0, 25, 5)]
def find_position(table, ch):
    if ch == "J":
        ch = "I"
    for i in range(5):
        for j in range(5):
            if table[i][j] == ch:
                return i, j
def prepare_text(text):
    text = ''.join(filter(str.isalpha, text.upper())).replace("J", "I")
    result = ""
    i = 0
    while i < len(text):
        a = text[i]
        if i + 1 < len(text):
            b = text[i + 1]
            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1
    return result
def encrypt(text, key):
    table = generate_key_table(key)
    text = prepare_text(text)
    cipher = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i + 1]
```

```
        r1, c1 = find_position(table, a)
        r2, c2 = find_position(table, b)
        if r1 == r2:
            cipher += table[r1][(c1 + 1) % 5]
            cipher += table[r2][(c2 + 1) % 5]
        elif c1 == c2:
            cipher += table[(r1 + 1) % 5][c1]
            cipher += table[(r2 + 1) % 5][c2]
        else:
            cipher += table[r1][c2]
            cipher += table[r2][c1]
    return cipher
def decrypt(cipher, key):
    table = generate_key_table(key)
    plain = ""
    for i in range(0, len(cipher), 2):
        a, b = cipher[i], cipher[i + 1]
        r1, c1 = find_position(table, a)
        r2, c2 = find_position(table, b)
        if r1 == r2:
            plain += table[r1][(c1 - 1) % 5]
            plain += table[r2][(c2 - 1) % 5]
        elif c1 == c2:
            plain += table[(r1 - 1) % 5][c1]
            plain += table[(r2 - 1) % 5][c2]
        else:
            plain += table[r1][c2]
            plain += table[r2][c1]
    return plain
key = input("Enter Key: ")
message = input("Enter Plaintext: ")
cipher = encrypt(message, key)
print("Encrypted:", cipher)
plain = decrypt(cipher, key)
print("Decrypted:", plain)
```

Output:

```
Enter Key: monarchy
Enter Plaintext: attack
Encrypted: RSSRDE
Decrypted: ATTACK
```

Result:
The Playfair Cipher algorithm was implemented successfully using a 5 × 5 key matrix. The plaintext was converted into pairs of letters and encrypted according to the same-row, same-column, and rectangle rules. The decryption process correctly recovered the prepared plaintext, demonstrating the successful implementation of Playfair encryption and decryption.

Conclusion:
The Playfair Cipher is a classical symmetric encryption technique that encrypts pairs of letters instead of individual letters. It provides better security than simple substitution ciphers such as the Caesar Cipher by reducing the effectiveness of frequency analysis. The experiment demonstrated the generation of a key matrix, preparation of plaintext into digraphs, and application of encryption and decryption rules. Although it is not suitable for modern secure communication, it is useful for understanding the basic concepts of classical cryptography.

(i) Hill Cipher
Description:
The Hill Cipher is a classical polygraphic substitution cipher that uses matrix multiplication and modular arithmetic to encrypt and decrypt messages. Unlike the Caesar and Playfair ciphers, the Hill Cipher encrypts a group of letters (blocks) together, making it more secure against frequency analysis.
In this implementation, a 3 × 3 key matrix is used. Each letter is first converted into its numerical equivalent (A = 0, B = 1, ..., Z = 25). The plaintext is divided into blocks of three characters, and if necessary, X is added as padding to complete the final block.
Encryption:
The plaintext vector is multiplied by the key matrix and the result is taken modulo 26.
Where:

$$C = (K \times P) \bmod 26$$

- C = Ciphertext vector
- K = Key Matrix
- P = Plaintext vector

Decryption:
The inverse of the key matrix modulo 26 is calculated and multiplied with the ciphertext vector.

$$P = (K^{-1} \times C) \bmod 26$$

The key matrix must be invertible modulo 26; otherwise, decryption is not possible.

Implementation:

```
import numpy as np
def text_to_numbers(text):
```

```
    text = text.upper().replace(" ", "")
    return [ord(c) - ord('A') for c in text]
def numbers_to_text(nums):
    return ''.join(chr((n % 26) + ord('A')) for n in nums)
def mod_inverse(a, m):
    a %= m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None
def inverse_key_matrix(K):
    det = int(round(np.linalg.det(K)))
    det_mod = det % 26
    inv_det = mod_inverse(det_mod, 26)
    if inv_det is None:
        raise ValueError("Key matrix is not invertible.")
    adj = np.round(det * np.linalg.inv(K)).astype(int)
    K_inv = (inv_det * adj) % 26
    return K_inv.astype(int)
def encrypt(plaintext, K):
    nums = text_to_numbers(plaintext)
    while len(nums) % 3 != 0:
        nums.append(ord('X') - ord('A'))
    cipher = []
    for i in range(0, len(nums), 3):
        block = np.array(nums[i:i+3]).reshape(3,1)
        enc = np.dot(K, block) % 26
        cipher.extend(enc.flatten())
    return numbers_to_text(cipher)
def decrypt(ciphertext, K):
    nums = text_to_numbers(ciphertext)
    K_inv = inverse_key_matrix(K)
    plain = []
    for i in range(0, len(nums), 3):
        block = np.array(nums[i:i+3]).reshape(3,1)
        dec = np.dot(K_inv, block) % 26
        plain.extend(dec.flatten())
    return numbers_to_text(plain)
K = np.array([
    [6,24,1],
    [13,16,10],
    [20,17,15]])
plaintext = "ACT"
```

```
cipher = encrypt(plaintext, K)
print("Encrypted :", cipher)
plain = decrypt(cipher, K)
print("Decrypted :", plain)
```

Output:

```
Encrypted : POH
Decrypted : ACT
```

Result:

The Hill Cipher algorithm was implemented successfully using a 3 × 3 key matrix. The plaintext "ACT" was encrypted into the ciphertext "POH", and the decryption process correctly recovered the original plaintext. The implementation demonstrated the use of matrix multiplication, modular arithmetic, and the inverse of the key matrix modulo 26 for secure encryption and decryption.

Conclusion:

The Hill Cipher is a powerful classical encryption technique that uses linear algebra concepts to encrypt groups of letters instead of individual characters. Compared to the Caesar and Playfair ciphers, it offers better security by encrypting multiple letters simultaneously. However, the security of the Hill Cipher depends on selecting a valid invertible key matrix. It is mainly used for educational purposes to understand matrix-based cryptography and modular arithmetic.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY స్వయం తేజస్విన్ భవ 1979